例 5-1 说明

例程文件夹：CH5EX01GD、CH5EX01HAL、CH5EX01STD

例 5-1 设计电路并编程实现：按 1Hz 规律控制 PE3 引脚上的 LED2 呼吸灯（心跳指示灯）闪烁；端口 PB0~PB7 上 8 个 LED 按移位点亮（即跑马灯、流水灯），循环显示。	数字资源 5-2 例 5-1 标准库 STD 法　　数字资源 5-3 例 5-1 硬件抽象库 HAL 法

硬件电路：如图 5-8 所示，STM32F407ZGT6 开发板上，PORT E 端口 PE3 连接 LED2、PORT B 端口中 PB0~PB7 连接 YLED0~7，阳极公共端接 3.3V。从图中分析，LED 是当 I/O 输出“0”时点亮，输出“1”是熄灭。GPIO 的属性设为通用输出、最大速度、上拉、推挽输出。

软件思路：定义心跳 LED 的名称为 LED2，定义 8 个 LED 的名称分别是 YLED0、YLED1、YLED2、YLED3、YLED4、YLED5、YLED6、YLED7。心跳指示灯的频率为 1Hz，跑马灯可按一定的频率顺序点亮。计时采用 SysTick 定时器延时获得。

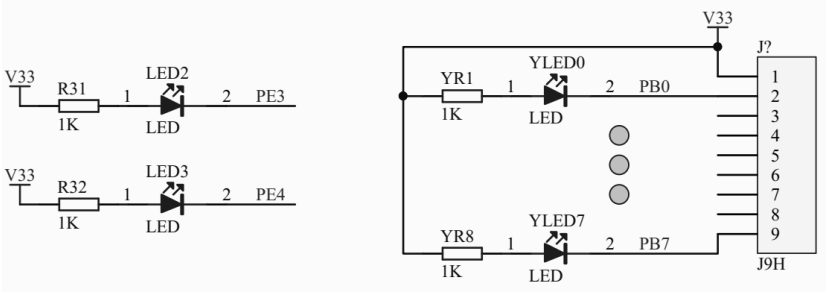


图 5-8 例 5-1 电路接口图

采用标准库 STD 法的流程是：首先依据模板新建工程，选定芯片型号，添加必要的库文件，分组管理这些文件；然后添加空白的 usermain.c 文件，在该文件中或新建.c/.h 文件中编写本题目所需 GPIO 端口的初始化函数 void LED_Init(void)和一些变量。最后在 usermain.c 文件修改 int main(void)函数，按初始化系统时钟、初始化 GPIO 外设、编写题目规定的功能等几个步骤去实现代码。编程完成后，进行调试、修改和观察。

管理工程时需加入的标准库文件至少有 startup_stm32f40_41xxx.s、stm32f4xx_rcc.c、stm32f4xx_gpio.c、stm32f4xx_syscfg.c 及其包含的头文件。

采用硬件抽象库 HAL 法的流程是：先运行 STM32CubeMX 软件，按题目要求完成系统时钟、GPIO 设置，生成 Keil MDK 工程。该工程已经具备了系统初始化的所有功能，读者在此基础上仅仅添加实现功能的代码，如本题的跑马灯按顺序点亮 LED。

打开 STM32CubeMX 软件（本书采用版本 V5.6.1），按例 5-1 硬件要求进行设置，如图 5-9 所示。图中可以看到，PE3 命名为 LED2，PE4 为 LED3，PB0~PB7 为 YLED0~7，属性均为通用输出、最大速度、上拉、推挽输出。在 Clock Configuration 栏进行时钟设置，时钟源为外部晶振 HSE（取值 8MHz），SYSCLK 为 168MHz。在系统设置 System Core 中，使能 SysTick，采用 SW 调试接口。

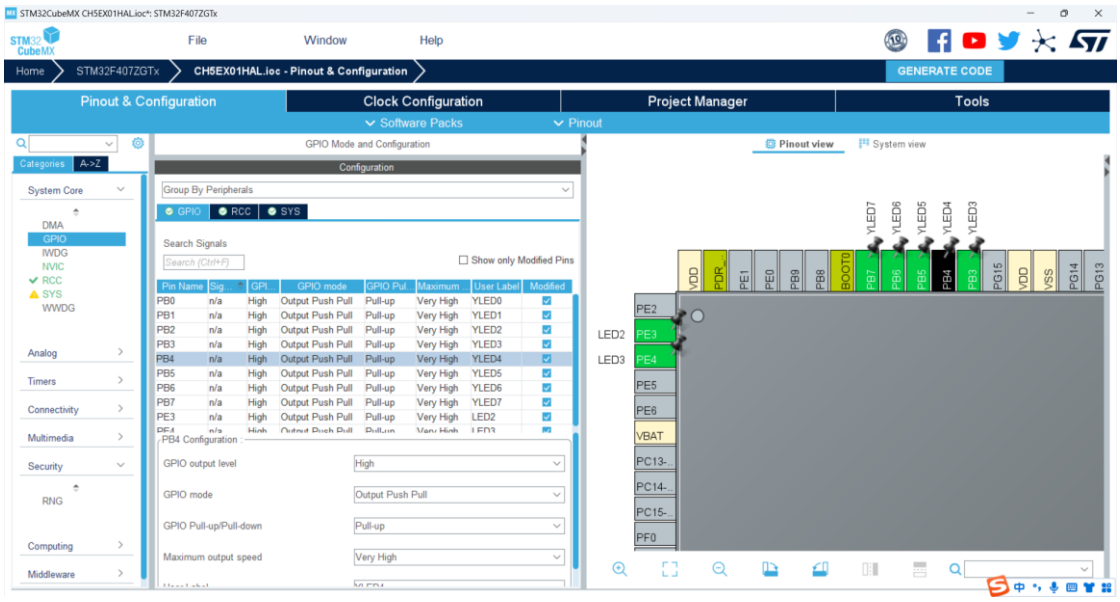


图 5-9 例 5-1 的 STM32CubeMX 设置

然后点击菜单“Generate Code”生成的 Keil MDK 工程文件，包含 Core、Drivers 和工程（名）三个文件夹。打开文件夹和里面的文件可以看出，外设驱动库函数、系统时钟、GPIO 外设的初始化都已存在，读者在此框架上仅需添加实现功能的代码。

代码展示：

文件	注释
工程文件列表 名称 CH5EX01HAL Core Drivers .mxproject CH5EX01HAL.ioc CH5EX01HAL.pdf CH5EX01HAL.txt	● Core 目录中包含本次工程的用户文件，如 main.c/.h、gpio.c/.h、rtc.c/.h、stm32f4xx_hal_msp.c、stm32f4xx_it.c/.h、system_stm32f4xx.c、stm32f4xx_hal_conf.h，都可以由用户修改 ● Drivers 目录中包含 STM32F4xx_HAL_Driver 库和 CMSIS 文件，共用文件 ● CH5EX01HAL 工程文件，含有 MDK-ARM 目录，包含.uvprojx 以及启动文件 startup_stm32f407xx.s ● CH5EX01HAL.ioc 文件为 STM32CubeMX 文件 CH5EX01HAL.pdf 和 txt 文件是本次配置的说明

需要编程的代码主要体现在 main.c/.h、gpio.c/.h 中，其中延时采用了 SysTick 定时器中断编写的 HAL_Delay( )函数，默认按 1ms 计数延时。操作 GPIO 时，HAL_GPIO_TogglePin( )函数用于翻转所选输出引脚的电平，HAL_GPIO_WritePin( )函数用于向某引脚写入高电平或低电平。

例 5-2 说明

例程文件夹：CH5EX02HAL、CH5EX02STD

例 5-2 设计电路并编程实现：通过 1 位按键控制 2 个 LED 亮灭。上电后 LED 熄灭；当按键第 1 次按下时 2 个 LED 点亮，第 2 次按下时其中 1 个 LED 亮，另 1 个 LED 灭，第 3 次交换点亮，第 4 次按下 2 个 LED 熄灭；周而复始。	数字资源 5-4 例 5-2 标准库 STD 法　　数字资源 5-5 例 5-2 硬件抽象库 HAL 法

硬件电路：如图 5-10 所示，STM32F407ZGT6 开发板上，PORT E 端口 PE3 连接 LED2、PE4 连接 LED3、PORT F 端口中 PF6、PF7 分别连接 S3 和 S4 按键。从图中分析，LED 是当 I/O 输出“0”时点亮，输出“1”是熄灭；按键常态为“1”，按下为“0”，低电平有效。GPIO 输出的属性设为通用输出、最大速度、上拉、推挽输出，输入的属性设为通用输入、上拉。

软件思路：分别定义两只 LED 的名称为 LED2、LED3，定义 2 个按键的名称分别是 KEY3、KEY4。采用查询的方式获得按键值，记录按键次数，再以此决定 LED 亮灭。

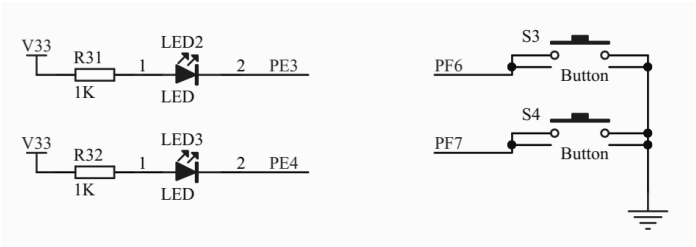


图 5-10 例 5-2 电路接口图

采用标准库 STD 法时，管理工程时需加入的驱动文件有 startup_stm32f40_41xxx.s、stm32f4xx_gpio.c、stm32f4xx_rcc.c、stm32f4xx_syscfg.c 及其包含的头文件。

采用硬件抽象库 HAL 法时，STM32CubeMX 设置的界面如图 5-11 所示，PE3、PE4 为输出、PF6、PF7 为输入。由向导生成工程后，在 gpio.c/.h 文件里有 LED 和按键的初始化函数 void MX_GPIO_Init(void)，可在 main.c 文件中直接调用。实现功能的代码主要体现 main 函数中，查询按键是否按下，并按条件分支对 LED 点亮或熄灭进行操作。

例 5-3 说明

例程文件夹：CH5EX03HAL、CH5EX03STD

例 5-3 采用外部中断实现例 5-2。通过 1 位按键控制 2 个 LED 亮灭。上电后 LED 熄灭；当按键第 1 次按下时 2 个 LED 点亮，第 2 按下时其中 1 个 LED 亮，另 1 个 LED 灭，第 3 次交换点亮，第 4 次按下 2 个 LED 熄灭。

数字资源 5-6 例 5-3 标准库 STD 法

数字资源 5-7 例 5-3 硬件抽象库 HAL 法

硬件电路：同例 5-2。

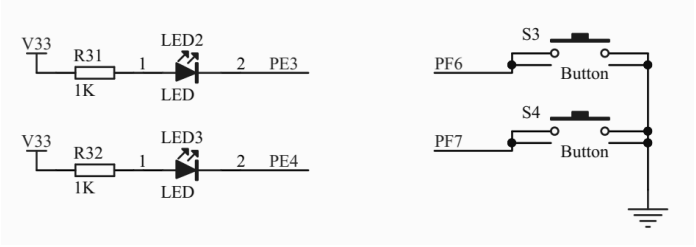


图 5-10 例 5-2 电路接口图

软件思路：配置 PE3、PE4 为输出模式；配置 PF6 为外部中断 EXTI6 下降沿触发、上拉模式，中断优先级分组为第 2 组，并配置抢占优先级和子优先级分别为 1。EXTI6 对应的中断服务函数为 void EXTI9_5_IRQHandler(void)，需要实现按键计数。该计数由主文件 main 函数中的 while 循环进行判断并分支使用。

采用标准库法时，需加入工程的标准库文件有 startup_stm32f40_41xxx.s 、misc.c/.h 、stm32f4xx_gpio.c/.h、stm32f4xx_rcc.c/.h、stm32f4xx_syscfg.c/.h、stm32f4xx_exti.c/.h。

采用硬件抽象库 HAL 法时，在 STM32CubeMX 中配置 PF6 为外部中断 EXTI6 下降沿触发、上拉模式，中断优先级为第 2 组，配置抢占优先级“1”和子优先级“1”，如图 5-16~17 所示。LED 和按键的初始化、外部中断设置函数均由 STM32CubeMX 生成，位于 gpio.c 文件。中断服务函数 void EXTI9_5_IRQHandler(void)在 stm32f4xx_it.c 文件中，用于处理按键次数 Index，而实现分支功能所需编写的代码在 main.c 文件中。在 main.c 中定义全局变量 Index，在 stm32f4xx_it.c 文件用 extern 外部使用。

例 6-1 说明

例程文件夹：CH6EX01HAL、CH6EX01STD

例 6-1 设计电路并编程实现：分别以 1Hz、10Hz 和 50Hz 的频率控制 PE3、PE4 引脚上的 LED 工作指示灯闪烁和 PE2 引脚上的蜂鸣器鸣叫。要求采用定时器中断的方法。（可以采用两个定时器）	数字资源 6-2 例 6-1 标准库 STD 法　数字资源 6-3 例 6-1 硬件抽象库 HAL 法

硬件电路：设计如图 6-26 所示，STM32F407ZGT6 开发板上，PORT E 端口 PE3 连接 LED2、PE4 连接 LED3，PE2 连接三极管电路以驱动蜂鸣器。LED 是当 I/O 输出“0”时点亮，输出“1”是熄灭。蜂鸣器是高电平时鸣叫。通用数字 I/O 的属性设为通用输出、最大速度、上拉、推挽输出。

软件思路：定义 LED 名称分别为 LED2、LED3，蜂鸣器名称为 BEEP。采用 SysTick 定时器控制 1Hz 工作指示灯 LED2（呼吸灯）。SysTick 为 1ms 时基，因此需要 1000 次中断。采用基本定时器 TIM6 控制 10Hz 指示灯 LED3 和 50Hz 蜂鸣器。按 50Hz，即 20ms 定时，已知初始参数 CK_INT=CK_PSC=84MHz，则设置分频数 TIM6_PSC=199，重载值 TIM6_ARR=8399，则单次定时 T_1=（199+1）×（8399+1）／84 000 000=20ms。20ms 中断 5 次可得到 10Hz 时间。

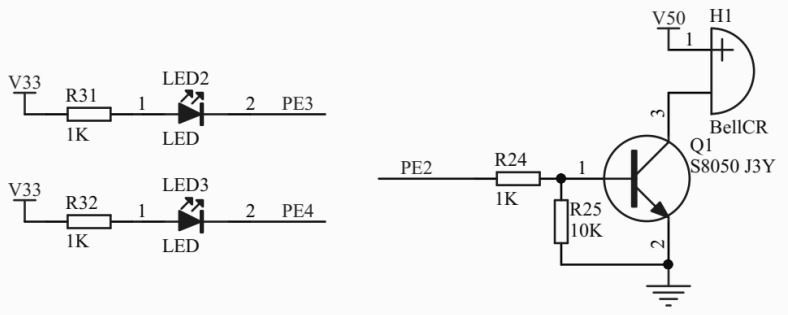


图 6-26 例 6-1 电路原理图

采用标准库法的流程是：首先依据模板新建工程，选定芯片型号，添加必要的库文件，分组管理这些文件；然后添加空白的 usermain.c 文件，在该文件中或新建.c/.h 文件中编写本题目所需的初始化函数 void LED_Init(void)、void TIM6_Init(void)和一些变量，并在文件 stm32f4xx_it.c 的 void SysTick_Handler(void)和 TIM6_DAC_IRQHandler 定时中断服务函数添加 LED 闪烁和蜂鸣器鸣叫代码。最后在 usermain.c 文件修改 int main(void)函数，按初始化系统时钟、初始化 GPIO、初始化 TIM6、开中断然后无限循环中等中断等几个步骤去实现代码。编程完成后，进行调试、修改和观察。

管理工程时需加入的标准库文件至少包括 startup_stm32f40_41xxx.s、stm32f4xx_rcc.c、stm32f4xx_gpio.c、stm32f4xx_syscfg.c、misc.c、stm32f4xx_tim.c 及其包含的头文件。

采用硬件抽象库 HAL 法的流程是：先运行 STM32CubeMX 软件，按题目要求完成系统时钟、GPIO 设置、TIM6 定时参数设置和允许中断，生成 Keil MDK 工程。TIM6 的参数如图 6-27 所示，PSC=199，ARR=8399，自动重载。TIM6 中断优先级设为第 2 组，抢占优先级 2，子优先级 0。在 main 函数中需要启动 TIM6，添加 HAL_TIM_Base_Start_IT(&htim6)语句。在中断文件 stm32f4xx_it.c 里的定时服务函数 void TIM6_DAC_IRQHandler(void)中填写 LED 和蜂鸣器翻转的代码。

例 6-2 说明

例程文件夹：CH6EX02HAL、CH6EX02STD

例 6-2 设计电路并编程实现：选用 STM32F407 芯片的 3 路 GPIO 引脚连接三色灯珠的红 R、绿 G、蓝 B 端，要求通过输出 3 路 PWM 波形分别调节 RGB 分量，实现颜色变化。（本题不考虑各色的组合比例，只实现颜色变化即可）	数字资源 6-4 例 6-2 标准库 STD 法　数字资源 6-5 例 6-2 硬件抽象库 HAL 法

硬件电路：设计如图 6-29 所示，STM32F407ZGT6 开发板上，PORT D 端口 PD13/14/15 外接三色灯珠的 R/G/B 端，共阳极接 3.3V。三色灯珠，也称为全彩 LED，其主要工作原理是：由红绿蓝三基色混色实现颜色的变化，采用输出波形的脉宽调制，即调节每色 LED 灯导通的占空比，在扫描速度很快的情况下，利用人眼的视觉惰性能展现颜色渐变的效果。设置通用数字 I/O 的属性为通用输出、最大速度、上拉、推挽输出。

软件思路：PD13/14/15 对应为定时器 TIM4 的 CH2/3/4 通道。采用通用定时器 TIM4 输出 PWM 波形，时基参数为内部时钟源 CK_INT=CK_PSC=84MHz，分频数 TIM4_PSC=99，重载值 TIM4_ARR=419，则单次定时 T_1=（99+1）×（419+1）／84 000 000=0.5ms。即 PWM 的频率为 2kHz。

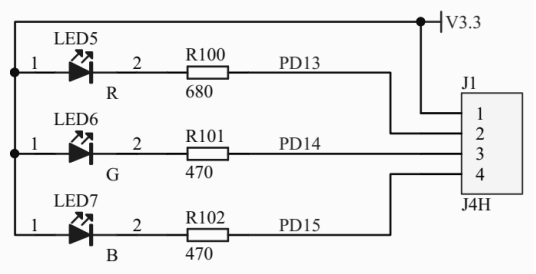




图 6-29 例 6-2 电路原理及接口图、实物照片

采用标准库法时，首先要初始化 TIM4 用于输出比较产生 3 路 PWM 的函数。然后在 usermain.c 文件修改 int main(void)函数，调用各初始化函数，并改变占空比更新 PWM 波形。管理工程时需加入的标准库文件至少包括 startup_stm32f40_41xxx.s、stm32f4xx_gpio.c、stm32f4xx_rcc.c、stm32f4xx_syscfg.c、stm32f4xx_tim.c 及其包含的头文件。

采用硬件抽象库 HAL 法时：运行 STM32CubeMX 软件，GPIO 设置 PE3 为 LED，通用输出、推挽、上拉、高速。TIM6 用于工作指示灯，参数设置 PSC=4999，ARR=8399，自动重载。TIM6 中断优先级设为第 2 组，抢占优先级 2，子优先级 0。TIM4 用于产生 PWM 波形，参数设置 PSC=19，ARR=8399，自动重载。配置 TIM4 的 CH2、CH3、CH4 通道为 PWM1 模式、使能，并设置占空比初值分别是 4200、200、8200。

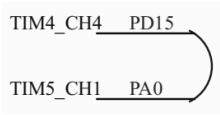
例 6-3 说明

例程文件夹：CH6EX03HAL、CH6EX03STD

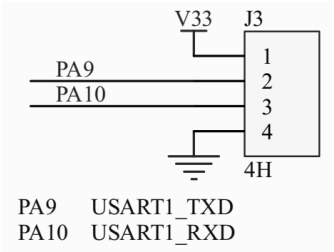
例 6-3 设计电路并编程实现：选用 STM32F407 芯片的 PA0 引脚测量输入波形的频率。采取 PD15 输出 PWM 波形，连接 PD15 和 PA0，改变 PWM 波形占空比和频率，观察测得值是否正确。并思考如何测量高电平的宽度？	数字资源 6-6 例 6-3 标准库 STD 法　数字资源 6-7 例 6-3 硬件抽象库 HAL 法

硬件电路：将 STM32F407ZGT6 开发板上的 GPIO A 端口的 PA0 与 PORT D 端口 PD15 连接起来。为观察方便，通过串口 USART1 外连 USB-TTL 模块，将测量值传输到电脑上，通过串口调试助手显示出来。电路接口如图 6-32 所示。

软件思路：PD15 为 TIM4 的 CH4，设置为 PWM1 模式，初始参数内部时钟源 CK_INT=CK_PSC=84MHz，分频数 TIM4_PSC=19，重载值 TIM4_ARR=839，则单次定时 T_1=（19+1）×（839+1）/ 84 000 000=0.2ms。即 PWM 的频率为 5kHz，占空比若数取为 210，按 PWM1 模式则占比 75%。TIM5 的分频数 TIM5_PSC=83，重载值 TIM5_ARR=4294967295，则捕捉频率为 1MHz。（注：TIM2 和 TIM5 是 32 位的定时器，ARR、CCR 和 CNT 寄存器是 32 位。另外，USART1 串口发送数据也可以采用 printf( )重定义函数，参见第 8.3.4 节。）



PD5 与 PA0 相连





USB-TTL 模块
RX 与 PA9 相连
TX 与 PA10 相连
GND 与 GND 连接

图 6-32 例 6-3 电路接口图

采用标准库法时，首先要自定义初始化 TIM4 用于通道 CH4 输出 1 路 PWM、TIM5 用于通道 CH1 的输入捕捉以及 USART1 串口的函数。然后在 usermain.c 文件添加中断服务函数 void TIM5_IRQHandler(void)，得到两次上升沿捕捉计数值。在主函数 int main(void)按序调用各初始化函数，并在 while 循环中通过中断返回的计数值计算出频率和周期，并由串口转发出去。

采用硬件抽象库 HAL 法时需要在 STM32CubeMX 软件中设置 TIM4、TIM5、USART1 等，参数同标准 STD 法，其中 TIM5 的通道 CH1 输入捕捉界面如图 6-33 所示。在 TIM5 输入捕捉的中断处理过程中，需要使用回调函数 void HAL_TIM_IC_CaptureCallback (TIM_HandleTypeDef *htim)获得计数值。因此须在 Project Manager→Advanced Settings 中使能 TIM 的 Register Callback 功能。

例 7-1 说明

例程文件夹：CH7EX01HAL、CH7EX01STD

例 7-1 请设计电路和编程完成 4 位动态共阳极数码管的显示。要求显示内容可变，同时 PE3 引脚上的工作指示灯 LED（呼吸灯）进行 1Hz 闪烁。

数字资源 7-2 例 7-1 标准库 STD 法

数字资源 7-3 例 7-1 硬件抽象库 HAL 法

硬件电路：设计如图 7-5 所示，STM32F407 开发板上外接数码管模块，GPIO B 端口 PB0~7 连接四位共阳极数码管的段选线，PB8~11 连接位选线，PNP 三极管发射极公共端接 3.3V。设置通用数字 I/O 的属性为通用输出、最大速度、上拉、推挽输出。由于共阳极端接 PNP 三极管反相了，所以位选线低电平有效，动态扫描时，PB8~11 依次为 0 才能顺序点亮四位数码管。

软件思路：由于 GPIOB 端口的 12 个引脚用于段码和位码数据输出，可以采用组合的方式构成端口 B 的 16 位数据，通过标准库中的 void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal)函数同时输出段选和位选信号。每次显示 4 位的动态扫描时间约为 20ms。显示内容是 4 位计数的大小。另外，TIM6 做为呼吸灯的定时器，按 1Hz 在进行中断。

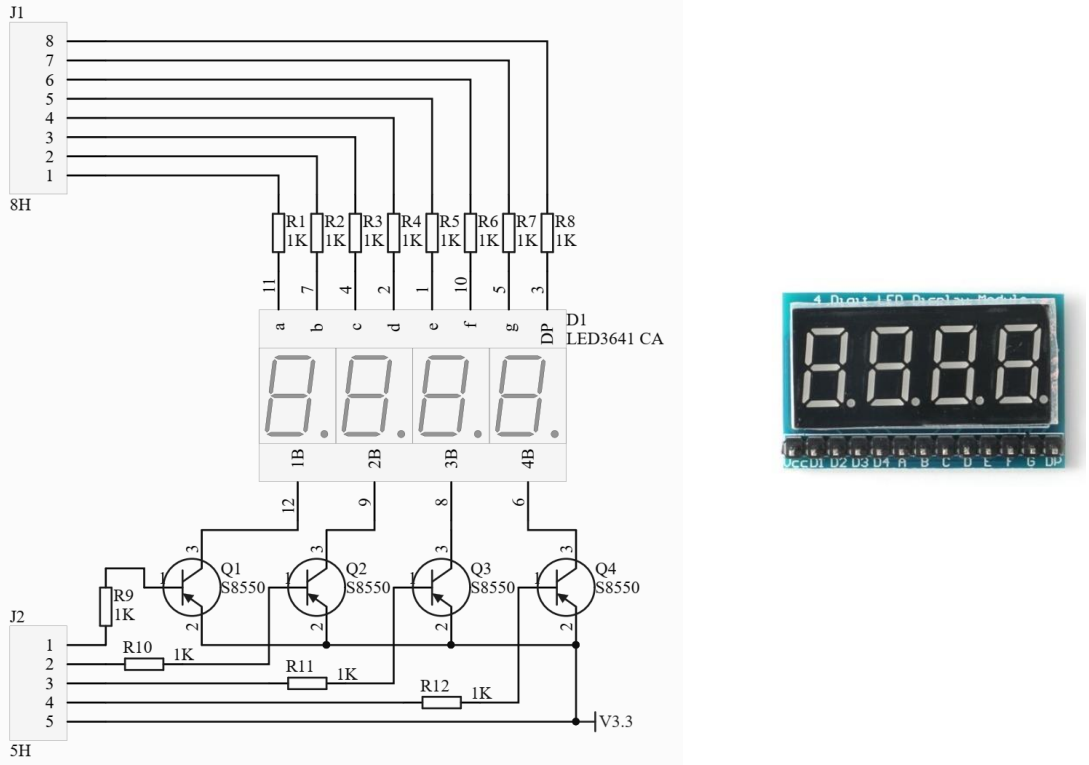


图 7-5 例 7-1 电路原理及接口图、实物照片

采用标准库法时，首先要初始化 GPIO PB0~11 引脚，编写 void LEDSHMG4_Init(void)函数，然后在 usermain.c 文件修改 int main(void)函数，调用各初始化函数，在 while 循环中改变显示数据，组合段码和位码进行显示，根据人眼视觉暂留现象，适当延时。另外，设置基本定时器 TIM6 为 1s 定时，控

制 LED 闪烁。

管理工程时需加入的标准库文件至少包括 startup_stm32f40_41xxx.s、stm32f4xx_rcc.c、misc.c、stm32f4xx_gpio.c、stm32f4xx_syscfg.c、stm32f4xx_tim.c 及其包含的头文件。

采用硬件抽象库 HAL 法时：运行 STM32CubeMX 软件，GPIO 设置 PB0~11、PE3 为普通输出，推挽、上拉、超高速。TIM6 用于工作指示灯产生 1s 定时，参数设置 PSC= 9999，ARR=8399，自动重载，中断优先级设为第 2 组，抢占优先级 2，子优先级 0。由于在 HAL 库中没有对 GPIO 端口多位的操作函数，故需转化为位操作，执行 void HAL_GPIO_WritePin (GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)。

例 7-2 说明

例程文件夹：CH7EX02HAL、CH7EX02STD

例 7-2 已知 TFT LCD 屏的规格是：2.8 寸、分辨率 320×240、RGB565、16 位色、驱动芯片 ILI9341、工作电压 3.3V/5V、背光电压 3.3V、并行 16 位数据总线、功耗 60~70mA、24 针接口。请设计 STM32F407 连接该屏的电路，并编程显示某固定图形和每隔 1s 增 1 的计数数字。

数字资源 7-4 例 7-2 标准库 STD 法

数字资源 7-5 例 7-2 硬件抽象库 HAL 法

硬件电路：如图 7-10 所示，STM32F407ZGT6 开发板上，通过 JLCD 端子外接 2.8 寸 TFT LCD 屏，接口对应关系如表 7-4 所示。由于连接 LCD 屏的片选为 NE4，故 HADDR[27:26]=11，扩展存储器地址范围为 0x6C00 0000~6FFF FFFF。地址总线中 A12 连接 LCD 屏的 RS 引脚，即 A12=RS=0 写命令，A12=RS=1 写数据。由于 TFT LCD 使用的是 16 位数据宽度，HADDR[0]并没有用到，只有 HADDR[25:1]是有效的，需要右移一位形成 A[24:0]，这样就得到 HADDR[13]=A12=RS，故 LCD 屏的命令口地址为 0x6C00 0000 或 0x6C00 1FFE，数据口地址为 0x6C00 2000。

表 7-4 STM32F407 与 LCD 屏的接口

MCU 接口	信号定义	LCD 屏接口	方向	分析
复用 PG2	A[25:0]	A12 接 RS	→	LCD 屏的命令/数据选择 RS=0 命令，RS=1 数据
复用 PD14/PD15/PD0/PD1/PE7/ PE8/PE9/PE10/PE11/PE12/PE13/ PE14/PE15/PD8/PD9/PD10	D[15:0]	接 DB0~15	⇆	双向数据总线，16 位
复用 PG12	NE[x]	NE4 接 CS	→	片选 4，BANK1 的子块 4
复用 PD4	NOE	接 RD	→	读外存储器信号
复用 PD5	NWE	接 WR	→	写外存储器信号
NRST	NRST	RESET	→	与 MCU 同时复位
PG7	LCD_BL	BL	→	LCD 屏背光控制

软件思路：根据硬件的分析，需要针对存储子块 4 的寄存器 FSMC_BCR4、FSMC_BTR4 和 FSMC_BWTR4 进行配置，编写初始化函数。初始化参数时需要匹配驱动芯片 ILI9341 的时序要求，设

置数据保持时间和地址建立时间。对于 LCD 屏的相关操作，借鉴厂家提供的 LCD.c/.h 文件。

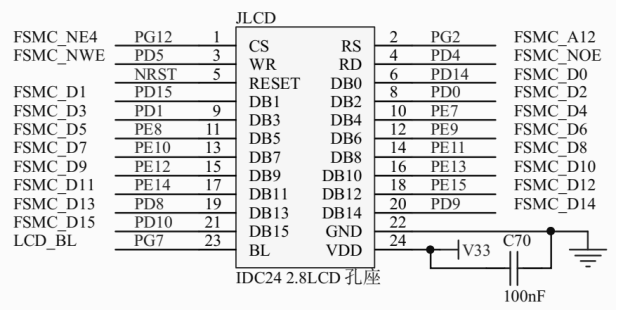


图 7-10 例 7-2 电路接口图和实物照片

采用标准库法时，首先要针对硬件接线完成配置 FSMC 的 void LCD_FSMC_Config(void) 函数的编程，然后根据驱动芯片的说明进行初始化 LCD，确认 LCD 屏存在。注意 FSMC 是挂在 AHB3 总线上的。初始化成功后调用 LCD 的操作函数，显示字符、图形和数字等。

管理工程时需加入的标准库文件至少包括 startup_stm32f40_41xxx.s 、stm32f4xx_rcc.c 、stm32f4xx_fsmc.c、stm32f4xx_gpio.c、stm32f4xx_syscfg.c 及其包含的头文件。

例 7-3 说明

例程文件夹：CH7EX03HAL、CH7EX03STD

例 7-3 已知 OLED 屏的规格是：0.96 寸 PMOLED、分辨率 128×64、白色、驱动芯片 SSD1306、工作电压 3.3V/5V、I²C 总线、功耗 0.06W（20mA）、4 针接口。请设计 STM32F407 连接该屏的电路，显示变化的计数值。

数字资源 7-6 例 7-3 标准库 STD 法

数字资源 7-7 例 7-3 硬件抽象库 HAL 法

硬件电路：如图 7-11 所示，STM32F407ZGT6 开发板上，将 GPIO F 端口的 PF0 和 PF1 设置为复用 I²C2 总线功能的 SDA 和 SCL，并与 OLED 屏对应的端子相连。同时需把 OLED 屏的 3.3V 电源和 GND 地接上。

软件思路：首先需要初始化 I²C 总线，通过 I²C 标准格式传输命令或者数据给 OLED 屏。对于 OLED 屏的相关操作，借鉴厂家提供的 OLED.c/.h 以及字库 font.h 文件。代码展示请参考第九章 I²C 总线相关部分内容，此处就不赘述了。（例 9-2）

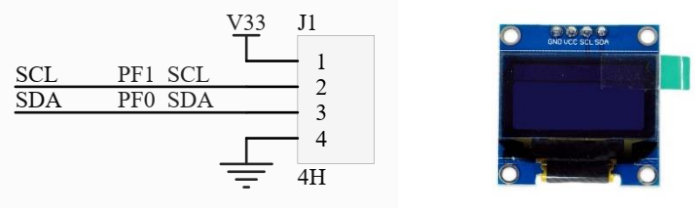


图 7-11 例 7-3 电路接口图和实物照片

例 8-1 说明

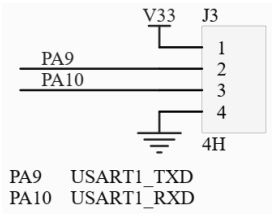
例程文件夹：CH8EX01HAL、CH8EX01STD

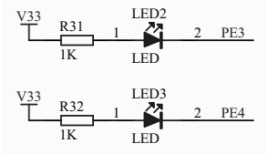
例 8-1 电脑采用 USB-TTL 模块与 STM32F407ZGT6 微控制器串口 USART1 通信：STM32F407 开发板按 2s 定时不停的发送计数值，由电脑端的串口助手显示出来；电脑端发命令让开发板上的 PE4 引脚的 LED3 亮/灭、PE2 引脚的蜂鸣器鸣叫/静默。	数字资源 8-2 例 8-1 标准库 STD 法　数字资源 8-3 例 8-1 硬件抽象库 HAL 法

硬件电路：如图 8-8 所示，STM32F407ZGT6 开发板上，PORT E 端口 PE3 连接 LED2 用于心跳闪烁（1Hz），PE4 连接 LED3，PE2 连接蜂鸣器驱动电路，PORT A 端口 PA9 和 PA10 连接 USB-TTL 模块的 RXD 和 TXD（注意要交叉，一方的发送端与另一方的接收端相连）用于全双工通信。



USB-TTL 模块





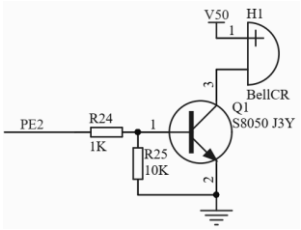


图 8-8 例 8-1 电路接口图

软件思路：根据电路可知采用了 5V/3.3VTTL UART 电平规范，设定通信参数为全双工异步串行通信，波特率 115200bps，字符帧格式为 10 位方式（1 位起始位+8 位数据位+1 位停止位），点对点，ASCII 码传送，STM32F407ZGT6 端以查询方式发送、中断方式接收。

制定电脑端发给芯片端的通信协议有 4 个字节，命令格式是：SD0E，SD1E，SB0E，SB1E。起始字符是 S，结尾是 E，中间 2 个字符 D0 表示 LED 灭，D1 表示 LED 亮，B0 表示蜂鸣器关，B1 表示蜂鸣器响。

采用标准库法时，首先对 USART1 进行初始化，复用 GPIO PA9 和 PA10，设置通信参数，并开启接收中断。然后在 USART1 中断服务函数 void USART1_IRQHandler(void)中，分析收到的报文，并通知 main 函数中的主程序处理报文，完成相应的功能。

管理工程时需加入的标准库文件至少有 startup_stm32f40_41xxx.s 、stm32f4xx_rcc.c 、stm32f4xx_gpio.c、misc.c、stm32f4xx_syscfg.c、stm32f4xx_usart.c 及其包含的头文件。

采用硬件抽象库时，需要先在 STM32CubeMX 软件中设置 USART1 的参数，注意要使能 UART 的 Register Callback，然后在 main.c 和 stm32f4xx_it.c 文件中添加接收中断回调函数 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)的处理代码，并在 main 函数中初始化串口后使用 HAL_UART_Receive_IT( )开启串口接收中断。而且在每次回调函数中处理完接收数据后，需要再次开启中断 HAL_UART_Receive_IT( )。

例 9-1 说明

例程文件夹：CH9EX01HAL、CH9EX01STD

例 9-1 利用 STM32F407 芯片的 SPI1 接口与外围 NOR Flash 芯片 W25Q128JV 进行读写操作。W25Q128 的前 4096 字节存放系统参数，不能随意更改，随后的其他区域用于存放系统运行数据。在运行中每隔 60s 将系统需要存储的 128 字节数据写入 W25Q128 芯片后部分区域。如果储存区满则覆盖原先数据。当用户按下按键，读出最近存储的数据并通过串口显示出来。

数字资源 9-2 例 9-1 标准库 STD 法

数字资源 9-3 例 9-1 硬件抽象库 HAL 法

题目分析：查阅资料可知，W25Q128JV 的容量规格是 128M 位，相当于 16M 字节，标准 SPI 时钟频率可以达到 133MHz，支持 SPI 总线的模式 0 与模式 3，支持双线全双工，MSB 先行，8 位或 16 位数据帧长度。访问 W25Q128 需要执行其规定的指令，如 0x06 表示写允许，0x03 表示读数据等。

NOR Flash 芯片要求在写入数据前，必须对目标区域进行擦除操作，即把目标区域中的数据位擦除为 1。W25Q128 支持扇区擦除、块擦除及整片擦除，最小的擦除单位是扇区。W25Q128 将 16M 的容量分为 256 个块，每个块大小为 64K 字节；每个块又分为 16 个扇区，每个扇区为 4K 字节（4096B）。因此，写入数据前必须保证所写目标存储区域的数据都是 0xFF。为了保护原来扇区的数据，需要在 STM32F407 的 SRAM 中开辟一个至少 4K 的缓存区，用于暂存、组合为一个完整扇区的内容。

W25Q128JV 阵列被组织成 65536 个可编程页面，每个页面 256 字节。W25Q128 芯片写入数据时不能跨页写入，一次最多写入一页（256 字节）。本例中，从 4096B 以后的地址开始写入数据，每次写入 128 字节的时候需要把位置记住，存放在第 1 个扇区中，以备下次接着这个地址往后写入。

第 1 扇区，第 0、1 字节用于标识符，“0x55、0x66”，第 2、3 字节用于记录写入位置，第 4、5 字节用于备注异常。开机时这 6 字节需要取出，得到记录位置，而且在每次写 128 字节数据时更新。第 1 扇区的其余部分暂时不做安排。

硬件电路：如图 9-3 所示，STM32F407ZGT6 开发板上将 SPI1 的引脚与 W25Q128JV 一一相连，PB3/4/5 为 SPI 总线，PF11 为片选线。USART1 的 PA9 和 PA10 与 USB-TTL 模块相连。按键 PF6 设置为外部中断模式。

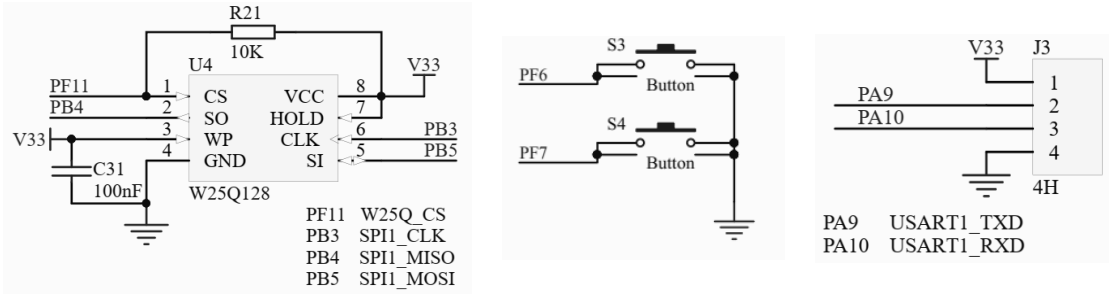


图 9-3 例 9-1 电路原理及接口图

软件思路：本例中，定义了 4K 缓存区为 W25Qx4K_BUFFER[4096]，记录次数的变量是 NN。W25Q128JV 的读写驱动函数由 w25qxx.c/.h 文件给出。初始化函数包括 USART1、按键 GPIO、按键 EXTI、SPI1。在主程序中，先判断芯片的 ID 号是否正确，然后根据记录次数和位置每隔 60s（实际演示是 5s）写入 128 字节的数据到 W25Q128JV 芯片。当按键按下，下降沿触发中断，在中断处理函数

中从 W25Q128JV 中读出最近写入的 128 字节数据，并由串口发送至电脑串口助手显示出来。写入的 128 个数据由 rand( )随机函数得到。

采用标准库方法管理工程时需加入的标准库文件至少有 startup_stm32f40_41xxx.s、stm32f4xx_rcc.c、stm32f4xx_gpio.c、stm32f4xx_exti.c、stm32f4xx_syscfg.c、stm32f4xx_spi.c、misc.c、stm32f4xx_usart.c 及其包含的头文件。

硬件抽象库法 HAL 与标准库法 STD 较为重要的区别在于 SPI 总线读写一个字节函数不一样，调用各自的库函数有所区别。从代码展示中可以看出 HAL 法比较简洁，这是由于调用了 HAL_SPI_TransmitReceive( ) 函数同时进行了收发，并把超时判断也封装进去了。

例 9-2 说明

例程文件夹：CH9EX02HAL、CH9EX02STD

例 9-2 参见例 7-3 和例 6-3，设计 STM32F407 芯片通过 I²C 总线接口与 OLED 屏相连形成一个显示系统。要求该系统能测量输入波形的频率，并将测量值显示在 0.96 寸 PMOLED 屏幕上。	数字资源 9-4 例 9-2 标准库 STD 法　数字资源 9-5 例 9-2 硬件抽象库 HAL 法

硬件电路：如图 9-9 所示，STM32F407ZGT6 开发板上，将 GPIO F 端口的 PF0 和 PF1 设置为复用 I²C2 总线的 SDA 和 SCL，并与 OLED 屏对应的端子相连。并把 TIM4 输出 PWM 波形的 CH4 通道，即 PD15 与 TIM5 输入捕捉 CH1 通道，即 PA0 相连。同时需把 OLED 屏的 3.3V 电源和 GND 地接上。

软件思路：首先需要初始化 I²C 总线，通过 I²C 标准格式传输命令或数据给 OLED 屏。对于 OLED 屏的相关操作，借鉴厂家提供的 OLED.c/.h 以及字库 font.h 文件。然后初始化 TIM4 和 TIM5，分别设置为 PWM 输出和输入捕捉。捕捉计数值由 void TIM5_IRQHandler(void)中断服务函数获得。在 main 函数中根据计数值计算出周期和频率，然后通过 OLED 显示出来。

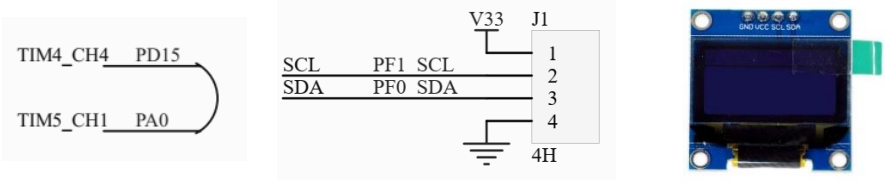


图 9-9 例 9-2 电路接口图和实物照片

采用标准库方法的操作 OLED 屏的主要代码集中在 OLED_I2C.c/.h 文件，由于 STMF4 库函数已经对通信时序封装好了，用户使用 I²C 功能非常方便，不再通过 GPIO 引脚模拟时序。因此主要编程工作体现在对 OLED 屏的命令控制、点阵坐标和字符图形的显示上。管理工程时需加入的标准库文件至少有 startup_stm32f40_41xxx.s 、 misc.c 、 stm32f4xx_rcc.c 、 stm32f4xx_gpio.c 、 stm32f4xx_syscfg.c 、 stm32f4xx_tim.c、stm32f4xx_i2c.c 及其包含的头文件。

代码中，I2C_WriteByte( )函数体现了一个完整 I²C 总线通信的发送步骤，对传输中的每步事件

（以 I2C_EVENT_为前缀）都要判断，以保证时序正确。

采用硬件抽象库 HAL 法时，关于 I²C 总线的读写基本操作函数与标准库 STD 法不一样，HAL 提供的函数 HAL_I2C_Mem_Write( )更加简洁，将 I²C 通信的事件处理、超时判断都封装好了，请读者对比有关代码。关于 I²C 总线的设置请参考图 9-10 所示。

例 9-3 说明

例程文件夹：CH9EX03HAL、CH9EX03STD

例 9-3 利用两块 STM32F407 开发板进行 CAN 通信实验。甲机通过按键触发发送一帧数据（8 个字节）给乙机，乙机将接收的数据显示出来。CAN 通信波特率设为 250kbps，扩展帧，发送的标识符 ID 是 0x0CF00400，数据场第 3 和 4 字节为发动机转速值，其他字节则自定数据大小。

数字资源 9-6 例 9-3 标准库 STD 法

数字资源 9-7 例 9-3 硬件抽象库 HAL 法

硬件电路：如图 9-15 所示，将两块 STM32F407ZGT6 开发板的 CAN1 接口 CANH 和 CANH、CANL 和 CANL 相连，PE3 上的 LED 指示接收和发送成功，按键 PF6 按下后启动发送。图中 TJA1040 为 CAN 总线收发器芯片，其作用是 CAN 收发信号 RX 和 TX 转换为总线信号。本次实验开发板需要 5V 或 12V 供电。显示的方法采用 FSMC 接口，这部分电路请参见第 7 章例 7-2。

软件思路：首先针对 GPIO、FSMC 和 CAN 进行初始化。CAN 的初始化主要是设置工作模式、波特率、滤波器等。采用查询方式检测按键是否按下，一旦按下按键则即刻发送 CAN 报文；采用中断方式接收 CAN 报文，通过全局变量传送到 main 函数进行显示。有关 CAN 初始化、发送和接收的函数放在 CAN407.c/.h 文件，需添加到工程。

本题波特率的参数设置为 CAN_Prescaler=BRP[9:0]=11，CAN_BS1=TS1[3:0]=6，CAN_BS1=TS2[2:0]=5 时，CAN 通信的波特率为 42M/[(1+7+6)*12]=250kbps。

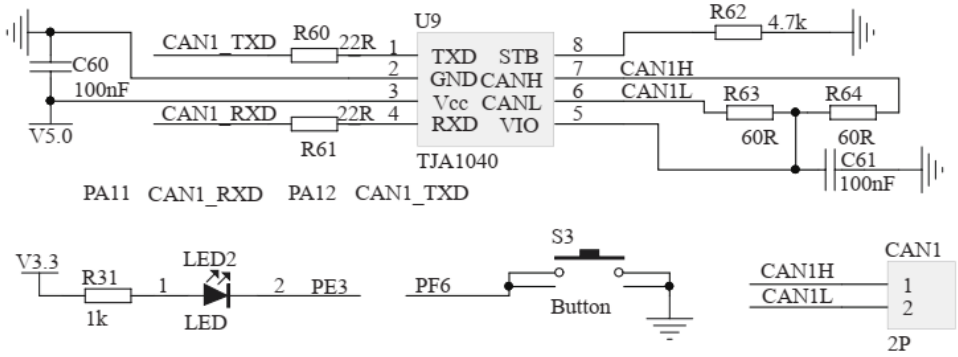


图 9-15 例 9-3 电路原理及接口图

采用标准库方法管理工程时需加入的标准库文件至少有 startup_stm32f40_41xxx.s、stm32f4xx_rcc.c、stm32f4xx_gpio.c、stm32f4xx_syscfg.c、stm32f4xx_fsmc.c、stm32f4xx_can.c、misc.c 及其包含的头文件。

在硬件抽象库 HAL 法中，通过 STM32CubeMX 设置 CAN 总线通信参数，如图 9-16 所示。生成工

程后，注意需要另行添加的代码有：初始化滤波器结构体 CAN_FilterTypeDef，调用函数 HAL_CAN_ConfigFilter( )进行配置；启动总线 HAL_CAN_Start(&hcan1)；激活接收中断 HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING) 。I 同时需在 STM32CubeMX 中配置 Register Callback 使能 CAN 总线回调函数。软件由回调函数 HAL_CAN_RxFifo0MsgPendingCallback( )来接收报文。为使用方便，自定义了发送 CAN 通信报文的函数 void My_CAN1_Send(uint8_t* txdata, uint8_t len)。

例 10-1 说明

例程文件夹：CHAEX01HAL、CHAEX01STD

例 10-1 需要采集 1 路模拟信号和芯片内部温度，请设计与 STM32F407ZGT6 的接口电路，并编程实现 A/D 采集以及显示测量结果。要求：ADC 工作在独立模式下，模拟信号由电位计调节，采用 4 位数码管分时显示电压（mV）和温度值（℃）。	数字资源 10-2 例 10-1 标准库 STD 法　数字资源 10-3 例 10-1 硬件抽象库 HAL 法

硬件电路：设计如图 10-2 所示，STM32F407ZGT6 开发板上，PORT B 端口 PB0~7 和 PB8~11 分别连接 LED 共阳极数码管的段选线和位选线（详细电路请参考图 7-5），PORT C 端口 PC5 为模拟信号输入引脚（即 ADC12_IN15），对外连接一只电位计输出端。

软件思路：对于共阳极 4 位数码管显示的编程请参考第 7 章的方法。ADC 部分按单次转换模式分别采集 2 路模拟信号。PC5 为 IN15 通道，内部温度传感器为 IN16 通道。main 函数中采用分时轮询的方法管理采集和显示。

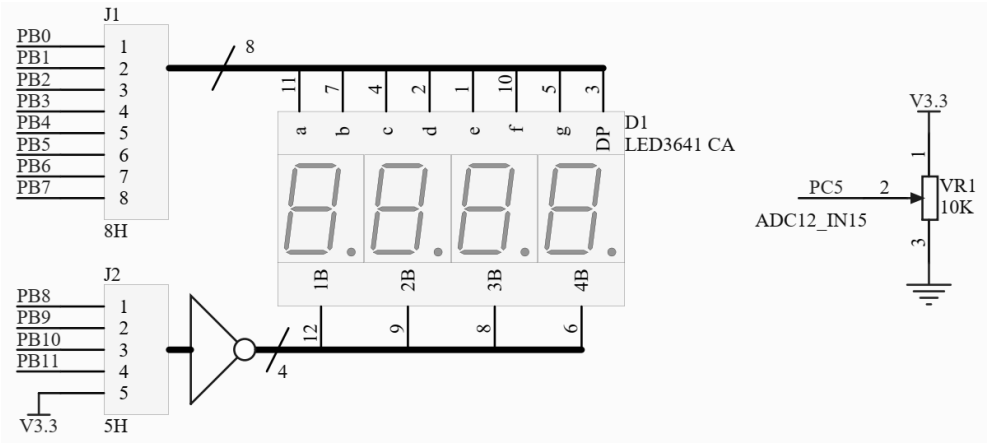


图 10-2 例 10-1 电路原理及接口图

采用标准库法的流程是：首先初始化配置 ADC 引脚 PC5 为模拟输入模式，使能 ADC1 时钟，配置通用 ADC1 为独立模式，采样率为 4 分频，12 位分辨率，单次转换模式，不需要外部触发；然后启动 ADC 转换，使能软件触发 ADC 转换。在 main 函数需先调用初始化数码管和 ADC1 的函数，然后在 while 循环中依次采集 IN15 和 IN16 共 2 通道，将结果适当转换后显示出来。

管理工程时需加入的标准库文件至少包括 startup_stm32f40_41xxx.s、stm32f4xx_rcc.c、stm32f4xx_gpio.c、stm32f4xx_syscfg.c、stm32f4xx_adc.c 及其包含的头文件。

本例如果 ADC 采用连续模式，转换结果数据使用中断方式读取，请读者思考如何编程。另外将在 while 循环中的采集并显示的语句如果写成一个函数来调用，怎么实现？

例 10-2 说明

例程文件夹：CHAEX02HAL、CHAEX02STD

例 10-2 请设计 STM32F407ZGT6 完成 6 路农田土壤信息传感器的模拟信号采集系统，采用常规组建立序列、使能 DMA 获取转换结果，并通过串口通信传送到电脑端查看。	数字资源 10–4 例 10–2 标准库 STD 法　数字资源 10–5 例 10–2 硬件抽象库 HAL 法

硬件电路：STM32F407ZGT6 开发板上，PORT A 端口 PA0~3 接入 4 路模拟信号，分别对应 ADC1 的 IN0~3 通道；2 路由 PORT C 端口 PC4 和 PC5，对应 ADC1 的 IN14 和 IN15 通道；另外附加 1 路是测量内部基准电压 V_{REFINT}（ADC1_IN17 通道）。串口通信采用 USART1 引脚 PORT A 端口 PA9 和 PA10，通过 USB-TTL 模块与电脑相连。

软件思路：6+1 路模拟输入通道都按常规组排序，依次是 IN0、IN1、IN2、IN3、IN14、IN15、IN17。配置 7 路通道采用函数 ADC_RegularChannelConfig( )完成，连续扫描转换模式。ADC1 所用的 DMA 功能属于 DMA2 Stream0 和 DMA_Channel_0 通道，方向为外设到内存，循环模式，按字对齐，采用 DMA_InitTypeDef 结构体进行初始化。初始化后可将 7 个转换结果 DR 直接存入数组变量中。

例 10-3 说明

例程文件夹：CHAEX03HAL、CHAEX03STD

例 10-3 请设计通过 STM32F407ZGT6 内部 DAC1 模块输出定值电压，DAC2 模块输出三角波的系统。DAC1 输出的电压可使用一个按键调整。DAC2 输出三角波的触发源由 TIM4 给定，频率由 TIM4 的时基决定，由另一个按键控制启停。

数字资源 10-6 例 10-3 标准库 STD 法

数字资源 10-7 例 10-3 硬件抽象库 HAL 法

硬件电路：STM32F407ZGT6 开发板中 DAC1 按 PA4 引脚输出电压和 DAC2 按 PA5 引脚产生三角波，通过万用表或示波器观察。PF6 和 PF7 为按键输入接口。通过按键 PF6 调整 DAC1 输出电压，按键 PF7 由按键奇偶次数启动和停止 DAC2 输出波形。电路接口如图 10-4 所示。

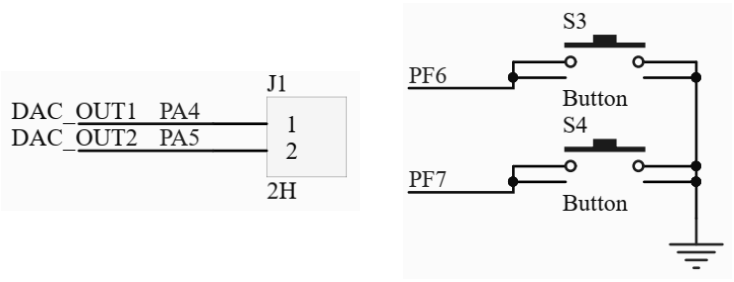


图 10-4 例 10-3 电路接口图

软件思路：应用 DAC 的操作步骤首先对 GPIO、DAC 和 TIM 初始化，使能 DAC 时钟、GPIO 数字输入时钟和模拟复用时钟、DAC 模拟复用引脚、2 个独立 DAC 通道模式、DAC1 为软件触发、DAC2 为定时器触发以及数据格式为 12 位右对齐等。TIM 的初始化主要是设置时基参数作为输出三角波的频率，并通过 TIM_SelectOutputTrigger( )函数产生 TRGO 触发事件。在主函数中，启动 DAC 转换，通过按键 PF6 调整输出电压并观察，通过按键 PF7 启停输出波形。

采用标准库方法管理工程时需加入的标准库文件至少包括 startup_stm32f40_41xxx.s、stm32f4xx_rcc.c、stm32f4xx_gpio.c、stm32f4xx_syscfg.c、stm32f4xx_dac.c、stm32f4xx_tim.c 及其包含的头文件。